Vulnerable Code Search: Transferable Attack for Code Language Models

Kaicheng Wang **Liyan Huang** **Jesse Thomason** **Weihang Wang**
University of Southern California
{wangkaic, liyanhua, jessetho, weihangw}@usc.edu

Abstract

Reliable code retrieval is crucial for developer productivity and effective code reuse. However, current neural code language models (CLMs) powering search tools are susceptible to adversarial attacks targeting non-functional textual elements. In this paper, we introduce a programming language-agnostic, transferable, adversarial attack that exploits this CLM vulnerability. Our approach perturbs identifiers within a code snippet without altering the snippet's functionality to artificially align the code with a target query. We demonstrate that our attack, even when computed using smaller code embedding models, such as CodeT5+, is highly effective and transferable to larger, closed-source embedding models, like Voyage-code-3, or LLMs like Gemini-3.1-Pro. Our attack can increase the similarity between the query and arbitrary, irrelevant code snippets, consequently degrading key retrieval metrics such as the Mean Reciprocal Rank (MRR) of state-of-the-art models by up to 77%. The experimental results highlight the fragility of current code search methods and underscore the need for more robust, semantic-aware approaches.

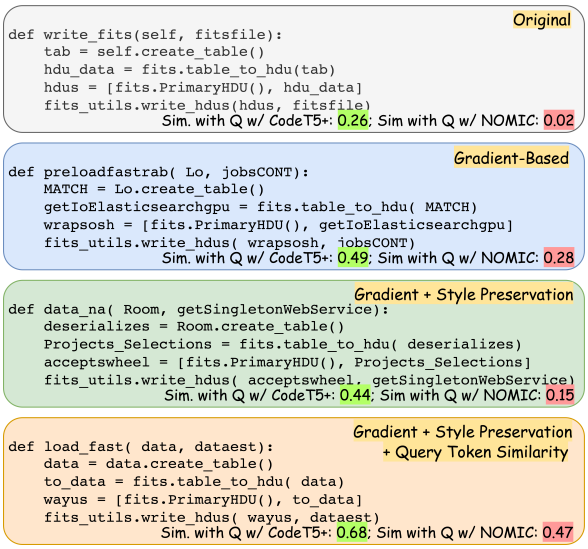


Figure 1: Overview of our attack for code search. Given the target query "`fastest way to load data`", the attack replaces identifiers in an unrelated code snippet while preserving functionality to deceive code search models. All attack variants are optimized on CodeT5+, but the resulting adversarial code snippets transfer beyond the surrogate model, also increasing similarity on the larger Nomic-embed-code model.

1 Introduction

The rapid expansion of the software ecosystem has intensified the need for efficient code retrieval to improve developers' productivity (Shekhar, 2024; Sun et al., 2024; Li et al., 2025). While the advent of Large Language Models (LLMs) and specialized Code Language Models (CLMs) has revolutionized tasks like repository-level reasoning (Jiang et al., 2026; Rozière et al., 2024; Hui et al., 2024; Chen et al., 2021; Liu et al., 2024a; Jimenez et al., 2024; Liu et al., 2024b), their computational cost often prohibits direct application in large-scale retrieval scenarios involving millions of candidates (Potvin and Levenberg, 2016; Howell et al., 2023). Consequently, for practical and efficient search, the industry continues relying on embedding-based methodologies, which encode queries and code into a shared latent space to rank candidates based on vector similarity (Di Grazia and Pradel, 2023; Liu et al., 2021).

However, embedding-based retrieval is susceptible to a critical vulnerability: modern CLMs are vulnerable to adversarial examples—code snippets containing small, functionality-preserving modifications that can drastically change the resulting embeddings (Chen et al., 2025; Qu et al., 2024; Wan et al., 2022). While most prior research has focused on adversarial attacks in classification tasks (Yefet et al., 2020; Zhou et al., 2022; Yao et al., 2024; Na et al., 2023), our work centers on code search, which leads to unique challenges. First, adversarial attacks in code search can be tailored to spe-

cific queries. By targeting specific queries, adversaries can have irrelevant or malicious code snippets crowding out legitimate results during certain task inquiries, subsequently influencing downstream coding decisions. Second, the typical code search workflow introduces additional robustness issues. Unlike classification, where malicious code is processed directly during inference and can be inspected by LLMs with reasoning abilities (Hort et al., 2025; Hossain et al., 2024; Jelodar et al., 2025), code search largely relies on offline embedding. When large code corpora are embedded offline, the modified code snippets maintain valid semantics and appear harmless. Since the subsequent retrieval process operates purely based on the similarity of these precomputed embeddings, detecting or mitigating these adversarial inputs becomes significantly more difficult.

In this paper, we demonstrate the shared vulnerability of the CLMs through transferable adversarial attacks. We generate adversarial examples by strategically replacing identifiers—user-defined symbols including function and variable names—within a code snippet to maximize the snippet's embedding similarity to a target query. The replacements are usually optimized on small white-box CLMs using gradient information, identifier style preservation, and query token similarity. As illustrated in Figure 1, although the attacks are crafted on `CodeT5+`, the resulting similarity improvement also appears under `Nomic-embed-code`, a model roughly 60 times larger. Based on our experiments, the attack transferability is broader: adversarial snippets generated on small surrogate models can transfer to larger models (e.g., `OASIS`, `Nomic-embed-code`) and closed-source systems, including embedding models (e.g., `Voyage-code-3`) and LLMs (e.g., `Gemini-3.1-Pro`) across five programming languages. Furthermore, the similarity change on the small source model strongly correlates with the changes on larger or closed-source target models, suggesting that attack effectiveness can be efficiently estimated.

Although state-of-the-art models achieve high scores on standard code search benchmarks (Li et al., 2024; Ott et al., 2022), they are vulnerable to our transferable adversarial attacks, leading to catastrophic drops in key retrieval metrics. For instance, we observed an absolute drop of up to 77% in Mean Reciprocal Rank (MRR) across all tested models on CosQA (Huang et al., 2021). The performance degradation exposes a critical gap: high benchmark scores do not necessarily reflect true semantic understanding of code. Instead, they highlight a reliance on brittle lexical features, underscoring the urgent need for improved model robustness.

In summary, our contributions are as follows:

- We propose an adversarial attack method for code search, which perturbs the code snippet to maximize its similarity with the target query while preserving functionality;
- We demonstrate the strong transferability of the attack: adversarial code snippets generated using small models can effectively deceive larger or closed-source systems; and
- We reveal that high benchmark scores mask a critical lack of robustness: current models prioritize superficial lexical matching over semantic reasoning, resulting in severe vulnerabilities that standard evaluations fail to capture.

2 Related Work

The field of Code Language Models (CLMs) has evolved rapidly, from early unified representations like CodeBERT (Feng et al., 2020) to encoder-decoders such as CodeT5 and CodeT5+ (Wang et al., 2021b, 2023). Subsequently, large generative CLMs such as Codex (Chen et al., 2021) and numerous open-source efforts, including CodeLlama (Rozière et al., 2024) and StarCoder (Li et al., 2023; Lozhkov et al., 2024), have demonstrated sophisticated code generation and reasoning abilities. The generative capabilities of CLMs have been further enhanced by Retrieval-Augmented Generation (RAG) (Zhao et al., 2024). In RAG systems, CLMs leverage efficiently retrieved code snippets—often sourced via embedding models—to improve contextual relevance and accuracy for complex tasks (Wang et al., 2025b; Chen et al., 2024; Wang et al., 2025a; Wu et al., 2024). The robustness of these underlying code embedding models is therefore critical, as the vulnerabilities explored in this paper directly threaten downstream applications.

Code search, a crucial task for efficient software development and code reuse (Nie et al., 2016), relies on embedding models for effective retrieval from large codebases. CodeSearchNet (Wu and Yan, 2022) established early benchmarks, with models like CodeBERT (Feng et al., 2020) learning joint natural language-code representations. To

enhance understanding, later work incorporated structural information: GraphCodeBERT (Guo et al., 2021) used data flow graphs, while UniXcoder (Guo et al., 2022) and SynCoBERT (Wang et al., 2021a) leveraged Abstract Syntax Trees. Later, contrastive learning, as seen in ContraCode (Jain et al., 2021), became a dominant training technique. Other strategies include adapting general CLM embeddings (Wang et al., 2023), fine-tuning LLMs (Nomic Team, 2025), or training on augmented data (Gao et al., 2025). Recently, black-box embedding services provided by OpenAI (OpenAI, 2024) and Voyage AI (Voyage AI, 2024) have also achieved state-of-the-art performance. However, our experimental results demonstrate that high benchmark scores do not guarantee robustness; top-performing code search models remain susceptible to the proposed adversarial attacks.

Applying gradient-based attack methods (Goodfellow et al., 2015) directly to natural language is challenging due to the discrete nature of tokens, making it difficult to apply small perturbations while maintaining syntactic and semantic integrity (Zhang et al., 2020b). Programming languages, however, provide unique opportunities for adversarial attacks through semantic-preserving transformations (Hort et al., 2025). Attack strategies vary based on the assumed knowledge of the target model. White-box attacks require access to model gradients. Methods like DAMP (Yefet et al., 2020), MHM (Zhang et al., 2020a), and GraphCodeAttack (Nguyen et al., 2024) leverage gradient signals to guide modifications aimed at causing misclassification. Without access to internal model information, black-box attacks employ various optimization techniques. For instance, CARL (Yao et al., 2024) and ALERT (Yang et al., 2022) apply reinforcement learning and genetic algorithms, respectively. Meanwhile, CodeAttack (Jha and Reddy, 2023) relies on repetitive querying to pinpoint vulnerable tokens, and Wen et al. (2025) investigates the efficacy of combined search strategies. Other black-box methods rely on heuristics, such as inserting comments or dead code (Na et al., 2023). Another emerging approach uses generative models to directly produce adversarial code examples, as explored by CBA (Zhang et al., 2024) and ITGen (Huang et al., 2025). In this work, we propose a novel adversarial attack method that leverages white-box attack techniques on one code search model to derive examples that also serve as transferable black-box attacks on other models.

3 Adversarial Attack for Code Search

In this section, we introduce our adversarial attack method against code search models. We first outline the threat model underlying our approach (Section 3.1). Subsequently, we detail the proposed methodology, including the use of gradient information to search for replacements (Section 3.2), the strategy for preserving identifier style (Section 3.3), and the incorporation of query token similarity (Section 3.4). Finally, we discuss the application of our method in black-box scenarios via attack transfer (Section 3.5).

3.1 Threat Model

Attack Goals: Given a natural language query Q and a target code snippet C (e.g., malicious or semantically unrelated to Q), the adversary aims to manipulate the retrieval ranking by generating an adversarial variant C' optimized to rank within the top-k retrieval results for Q.

Attacker Knowledge: The adversary has full white-box access to a code embedding model, referred to as the *surrogate model*. The surrogate model may differ from the actual victim model used by the code search system. The attacker can use the surrogate model to compute the similarity between Q and C, along with the gradient of the similarity score with respect to the input token embeddings, to guide the generation of the adversarial example.

Attacker Capability & Practicality: The attack targets the indexing phase, requiring the adversary to inject or modify code in the retrieval corpus. This is realistic in open-source settings, where corpus poisoning and software supply-chain attacks can occur through fraudulent repositories, malicious commits, or compromised maintainers (Wan et al., 2022). Once such code is indexed, the adversarial variant C' can influence future retrieval results.

Attack Constraints: To ensure the attack remains functional, code modification must not alter the execution logic of the target code C. Semantic preservation of the code snippet is ensured by maintaining the Abstract Syntax Tree (AST) structure unchanged before and after the attack.

3.2 Gradient-Guided Optimization

Gradient-Based Approximation. Let Q_{emb} and C_{emb} denote the initial embeddings for query Q and code snippet C after tokenization and mapping.

We define the similarity score as $\text{Sim}_\theta(Q, C) = G_\theta(Q_{\text{emb}}, C_{\text{emb}})$, where θ denotes the surrogate model parameters. Our goal is to find a modified snippet C' that maximizes the similarity $\text{Sim}_\theta(Q, C')$.

Let $\delta = C'_{\text{emb}} - C_{\text{emb}}$. By applying a first-order Taylor expansion around C_{emb}, the similarity change can be approximated as:

$$\begin{aligned}\Delta\text{Sim}_\theta &= G_\theta(Q_{\text{emb}}, C'_{\text{emb}}) - G_\theta(Q_{\text{emb}}, C_{\text{emb}}) \\ &\approx \delta^\top \nabla_{C_{\text{emb}}} G_\theta(Q_{\text{emb}}, C_{\text{emb}}).\end{aligned} \tag{1}$$

Here, the zeroth-order terms $G_\theta(Q_{\text{emb}}, C_{\text{emb}})$ cancel out, leaving the gradient term to approximate the change. The gradient $\nabla_{C_{\text{emb}}} G_\theta(Q_{\text{emb}}, C_{\text{emb}})$ is a matrix of shape $c \times d$, where c is the context length and d is the embedding dimension.

Greedy Search. Consider an identifier w in the code snippet C. Let t be a token in w, and let $\mathcal{P}_t = \{p_1, \ldots, p_k\}$ denote the set of indices where t appears as part of the identifier w in the input token sequence. Note that $\mathcal{P}_t$ excludes positions where t appears as a standalone keyword or within other identifiers.

When substituting t with a candidate token $t' \in V$, the perturbation vector δ is sparse. It is non-zero only at indices $i \in \mathcal{P}_t$, where each entry corresponds to the difference between the candidate and original token embeddings $\mathbf{e}_{t'}$ and $\mathbf{e}_t$:

$$\delta[i] = \begin{cases} \mathbf{e}_{t'} - \mathbf{e}_t & \text{if } i \in \mathcal{P}_t, \\ 0 & \text{otherwise.} \end{cases}$$

By substituting the sparse δ into Eq. 1, we isolate the contribution of this token replacement. Since the gradient at index i, denoted $\nabla_{C_{\text{emb}}} G_\theta(Q_{\text{emb}}, C_{\text{emb}})[i]$, is a vector of shape $d \times 1$, we can compute the scalar contribution via a dot product with the $1 \times d$ perturbation vector $(\mathbf{e}_{t'} - \mathbf{e}_t)^\top$. We define the total contribution as the *influence*:

$$\begin{aligned}&\textit{influence}(t, t', \mathcal{P}_t) \\ &= (\mathbf{e}_{t'} - \mathbf{e}_t)^\top \left(\sum_{i \in \mathcal{P}_t} \nabla_{C_{\text{emb}}} G_\theta(Q_{\text{emb}}, C_{\text{emb}})[i] \right),\end{aligned} \tag{2}$$

where the summation aggregates the gradient values over all occurrences of the target token in w.

Finally, let V be the set of identifier-compatible tokens within the model's vocabulary. We identify the optimal replacement t^* by searching for the candidate that maximizes the influence:

$$t^* = \underset{t' \in V}{\text{argmax}} \ \textit{influence}(t, t', \mathcal{P}_t).$$

Crucially, under the first-order approximation, the perturbations operate in orthogonal subspaces because the position sets $\mathcal{P}_t$ for distinct tokens are disjoint. The independence allows us to optimize each token replacement via a greedy approach without requiring combinatorial optimization.

To ensure the attack preserves the code's execution logic, we also enforce two renaming constraints: 1) **Consistency**: Every occurrence of a specific identifier w is replaced by the same new identifier w'; and 2) **Uniqueness**: Distinct identifiers are replaced by distinct new identifiers. Implementation details for these constraints are provided in Appendix D.

3.3 Identifier Style Constraints

As illustrated in Figure 1, naive application of gradient-based optimization often yields long, unintelligible identifiers. Such anomalies make the adversarial code easily detectable by both human programmers and coding agents. To mitigate this, we enforce style-consistency constraints designed to preserve common naming conventions like `camelCase` and `snake_case`. We therefore filter the search space to ensure that valid replacements preserve the original style by including at least the same number of underscores or uppercase letters. Additionally, tokens serving solely as structural separators (e.g., standalone underscores) are irreplaceable. Empirically, we find that enforcing these constraints improves the visual naturalness and plausibility of the code with a marginal reduction in attack efficacy.

3.4 Query Token Similarity

We observe that code search models often exhibit a bias towards snippets that share explicit textual overlap with the target query. To exploit this property, we augment the gradient-based selection objective with a term that rewards proximity to query tokens. Specifically, when considering a candidate replacement token t', we also calculate its similarity to the most aligned token within the query Q. With the refined objective, we find the optimal

replacement token t^* as

$$t^* = \underset{t' \in V}{\operatorname{argmax}} \left[\textit{influence}(t, t', \mathcal{P}_t) + \alpha \max_{q \in Q} (\mathbf{e}_q^\top \mathbf{e}_{t'}) \right], \tag{3}$$

where $\mathbf{e}_q$ and $\mathbf{e}_{t'}$ represent the embeddings of a query token q and the candidate replacement t', respectively. Here, the search space V is also restricted to candidates that satisfy the style constraints detailed in Section 3.3. The term $\max_{q \in Q}(\mathbf{e}_q^\top \mathbf{e}_{t'})$ acts as a guidance signal, encouraging the selection of tokens that are similar in embedding space to tokens in the query. We introduce a hyperparameter α to balance the gradient-based influence and this query token similarity regularizer. Empirically, we set $\alpha = 0.1$; detailed ablation studies justifying this choice are provided in Appendix E.10.

3.5 Attack Transfer

The proposed attack framework requires white-box access to model parameters and sufficient computational resources to back-propagate gradients and compute token similarities. These prerequisites may not be met in real-world scenarios, particularly when targeting large-scale or proprietary black-box code search systems.

To address this, we introduce Attack Transfer. Consider a target model with parameters θ^*, which is inaccessible or too computationally expensive to attack directly. We can instead generate the adversarial snippet $C' = \textsc{AdvAttack}(Q, \theta)$ using a smaller, accessible surrogate model with parameters θ. Our method relies on the observation that adversarial examples are usually model-agnostic: that is, an optimization that maximizes $\text{Sim}_\theta(Q, C')$ is likely to also have high $\text{Sim}_{\theta^*}(Q, C')$. The cross-model transferability allows us to attack complex victim models by optimizing solely on the surrogate.

4 Experiments and Results

In this section, we evaluate our adversarial attack across embedding models, code search benchmarks, and LLM-based retrieval. We first quantify the attack's effectiveness and cross-model transferability (Section 4.1) and compare it with existing baselines (Section 4.2). We then examine the attack's impact on code search benchmarks in both per-query and shared-corpus settings (Sections 4.3 and 4.4) and evaluate the attack's transferability to LLM-based retrieval (Section 4.5). Finally, we investigate finetuning-based defenses (Section 4.6) and include an ablation study about each individual attack component (Section 4.7).

Unless otherwise noted, adversarial attacks are applied to *(query, code)* pairs. For each pair, we execute our adversarial attack for 5 iterations, creating a candidate set that includes each iteration's output and the original code. From the set, we select the snippet with the highest similarity score to the target query as the final adversarial example. In other words, when no perturbation yields a positive shift in similarity, the original code snippet is preserved, resulting in zero change. The shared-corpus experiment additionally considers adversarial snippets optimized for multiple queries at once. The design choices are justified in Appendix E.10.

Models. We generate attacks using two surrogate embedding models, `CodeT5+` (Wang et al., 2023) and `OASIS` (Gao et al., 2025), and evaluate transfer to `Nomic-embed-code` (Nomic Team, 2025), `Voyage-code-3` (Voyage AI, 2024), and the generative models `GPT-5.4-mini` and `Gemini-3.1-Pro`. Model details are provided in Appendix C.

Datasets. We use CosQA (Huang et al., 2021) and CLARC (Wang et al., 2026) to evaluate attack effectiveness, transferability, and code-search degradation, and RepoQA (Liu et al., 2024a) to measure the impact on generative-model retrieval. HumanEval-X (Zheng et al., 2023) is used only for the cross-language analysis in the appendix. Dataset statistics are provided in Table 10.

4.1 Effectiveness and Transferability

We evaluate attack effectiveness on 20,000 *(query, code)* pairs sampled from CosQA (Huang et al., 2021) and CLARC (Wang et al., 2026). Attacks are generated on `CodeT5+` or `OASIS` as Surrogate Models and then evaluated on multiple Eval Models in Table 1. Metric details are provided in Appendix E.2.

Effectiveness on Surrogate Models. The shaded rows show that the attack increases query-code similarity for over 97% of examples on the Surrogate Model. The gains are generally larger for `CodeT5+` than for `OASIS`, which may reflect the denser embedding space of `OASIS` (Appendix E.1) and its robustness-oriented training (Gao et al., 2025). The attack is also stronger on CosQA than on CLARC, suggesting that dataset or language characteristics affect attack magnitude.

Attack Transferability. The unshaded rows

Table 1: Similarity Changes and Transferability. The shaded rows indicate identical Surrogate and Eval models. The substantial similarity increases (Δ Sim) observed across all rows demonstrate both high attack effectiveness and robust transferability to diverse Eval models. High precision and strong Pearson (r) and Spearman (ρ) correlation coefficients further confirm that similarity improvements reliably transfer. Δ Sim, r, and ρ are scaled by 100.

Dataset	Surrogate Model	Eval Model	Δ Sim.	Improved Counts	Unimproved Counts	Precision	r	ρ
CosQA (Python)	CodeT5+	CodeT5+	38.86 ± 10.44	9899	101	-	-	-
		OASIS	18.15 ± 6.03	9898	102	99.99	63.37	58.69
		Nomic-embed-code	39.25 ± 10.89	9895	105	99.96	62.34	55.89
		Voyage-code-3	27.13 ± 8.96	9896	104	99.97	64.24	59.93
	OASIS	CodeT5+	23.23 ± 12.57	9670	330	98.01	75.56	75.49
		OASIS	13.68 ± 6.89	9866	134	-	-	-
		Nomic-embed-code	28.37 ± 12.91	9803	197	99.36	84.57	83.87
		Voyage-code-3	19.04 ± 10.08	9810	190	99.37	88.40	88.09
CLARC (C++)	CodeT5+	CodeT5+	26.60 ± 9.13	9866	134	-	-	-
		OASIS	13.10 ± 5.75	9825	175	99.64	60.70	55.61
		Nomic-embed-code	24.66 ± 9.94	9797	203	99.64	62.53	56.34
		Voyage-code-3	13.96 ± 6.99	9854	146	99.67	58.49	54.53
	OASIS	CodeT5+	12.78 ± 8.79	9360	640	95.72	69.21	70.79
		OASIS	9.21 ± 5.93	9741	259	-	-	-
		Nomic-embed-code	15.79 ± 9.89	9607	393	98.64	87.22	88.38
		Voyage-code-3	8.80 ± 6.55	9510	490	96.57	85.85	86.74

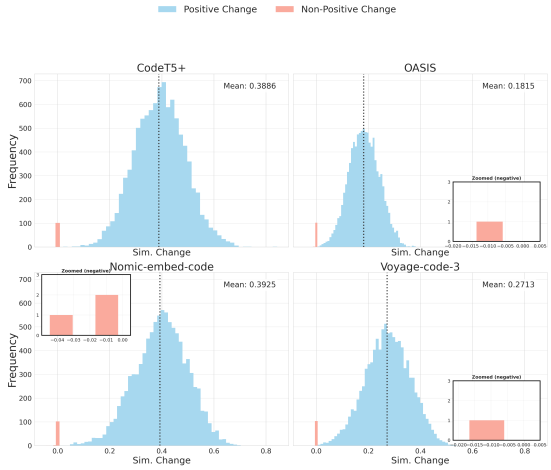


Figure 2: Distribution of Code Similarity Changes on CosQA. The Surrogate Model is `CodeT5+`, and the Eval Models are labeled at the top of each subplot. Negative similarity changes are smaller in magnitude than positive ones.

show strong transfer to diverse Eval Models, including robustness-enhanced, finetuned, and closed-source systems. Across Surrogate/Eval pairs, the attack increases similarity in over 95% of cases, with high precision and positive Pearson/Spearman correlations indicating that surrogate gains generally predict transfer gains. When transfer fails, the negative shifts are small compared to successful gains (Figure 2 and Appendix E.3).

Tokenizer differences do not block transferability. Although `CodeT5+` has a much smaller vocabulary than `Nomic-embed-code` or `Voyage-code-3`, it still transfers effectively to these larger models. Overall, `CodeT5+`-based attacks tend to produce larger Δ Sim, while `OASIS`-based attacks yield more consistent correlations.

Beyond cross-model transfer, the attack also transfers across queries sharing the same intents. As detailed in Appendix E.4, adversarial snippets optimized for one query remain effective for semantically equivalent paraphrases, suggesting that the attack targets the underlying intent rather than the specific target query string.

4.2 Comparison with Baselines

We compare Attack Transfer with two baseline attacks on 1,000 *(query, code)* pairs from CosQA. In the white-box setting, we transfer attacks generated on `CodeT5+` to `OASIS` and compare against DAMP (Yefet et al., 2020) applied directly to `OASIS`. In the black-box setting, we transfer attacks from `CodeT5+` to `Voyage-code-3` and compare against CodeAttack (Jha and Reddy, 2023). Since DAMP and CodeAttack were originally designed for code classification, we adapt them to the code search setting.

Table 2 shows that our transferred attack yields stronger effectiveness and better efficiency. In the white-box setting, Attack Transfer produces a larger Δ Sim. than DAMP while using less GPU time. In the black-box setting, the gap is larger: our transferred attack achieves about a $5\times$ higher Δ Sim. than CodeAttack while reducing the required queries from $\sim$10.8M API calls to 1k calls. The baseline attacks also transfer less effectively across models, as detailed in Appendix E.5.

Table 2: Comparison of Attack Transfer against baselines. Our method achieves stronger attack effectiveness with substantially lower computational or query cost. GPU hours are calculated on a NVIDIA L40 ADA 48GB GPU.

Method	Δ Sim.	Improved Counts	Efficiency
White Box			
DAMP	10.45 ± 5.48	978	81 mins
Ours	18.66 ± 5.52	999	47 mins
Black Box			
CodeAttack	5.18 ± 4.18	907	10.8m Calls
Ours	28.03 ± 8.29	1,000	1k Calls

4.3 Application on Code Search Benchmarks

In this experiment, `CodeT5+` is used as the Surrogate Model. For each query in the CosQA and CLARC datasets, we select 10%[1] of irrelevant code snippets and apply our attack to maximize their similarity to the query. These adversarially modified snippets replace the original ones in the candidate pools. Eval Models (`CodeT5+`, `OASIS`, `Nomic-embed-code`, and `Voyage-code-3`) then embed the queries and the modified pools, and rerank the code snippets by similarity. The retrieval metrics are measured before and after the adversarial attack.

As shown in Table 3, replacing only 10% of irrelevant candidates with adversarial versions results in sharp drops across all models and retrieval metrics. Before the attack, models like `OASIS`, `Nomic-embed-code`, and `Voyage-code-3` achieve strong R@5 (>90%) and MRR (74-87%) scores, suggesting the task is nearly "solved." However, after the attack, MRR drops to single digits, and R@5 drops to below 15% for all Eval Models, revealing that high benchmark scores do not guarantee robustness against adversarial manipulation.

As expected, `CodeT5+`, the Surrogate Model, suffers severe degradation. Yet the attack also transfers effectively to other models. `OASIS` remains vulnerable, despite being trained against hard negatives with similar keywords on an augmented dataset. `Nomic-embed-code`'s substantial performance decrease indicates even large CLMs can overly depend on lexical features like identifiers, suggesting that scale-up does not naturally yield greater robustness. While `Voyage-code-3` is the most resilient when allowed to retrieve more candidates (e.g., Recall@20), it still suffers from a drastic drop in MRR and Recall@1, confirming that the attack effectively buries the ground truth beneath adversarial noise.

[1]The results when other ratios of the corpus are attacked are available in Appendix E.7.

Table 3: Code search degradation under a 10% adversarial attack on CosQA using `CodeT5+` as the surrogate model. The attack causes large drops across retrieval metrics and Eval Models. Full metrics are reported in Appendix E.7.

	CodeT5+			Nomic-embed-code		
Setting	MRR	NDCG	R@1	MRR	NDCG	R@1
Original	74.08	78.52	64.00	83.03	86.55	73.80
Adversarial	1.79	2.46	1.20	5.95	8.13	3.00
Δ	−72.29	−76.06	−62.80	−77.07	−78.42	−70.80
	OASIS			Voyage-code-3		
Setting	MRR	NDCG	R@1	MRR	NDCG	R@1
Original	80.57	84.73	71.00	87.06	89.90	79.40
Adversarial	5.99	8.86	2.60	9.52	13.27	4.40
Δ	−74.58	−75.87	−68.40	−77.54	−76.63	−75.00

4.4 Shared Corpus Attack

The experiments in Section 4.3 attack a separate candidate pool for each query. We now consider the more constrained setting in which all queries are retrieved against a single shared corpus, so that every adversarial snippet is simultaneously visible to all queries. We sample 20 queries and construct a corpus of 500 code snippets, consisting of the 20 ground-truth snippets and 480 irrelevant ones. All the queries and the code snippets are from CosQA.

We first randomly select 50 of the 480 irrelevant snippets and attack them with `CodeT5+` as the Surrogate Model, targeting each of the 20 queries. For every query, we retain only the single most effective adversarial snippet, i.e., the one with the highest `CodeT5+` similarity to that query. Inserting these snippets into the corpus yields the "20 attacked snippets" setting, in which each query is targeted by exactly one adversarial snippet.

To further reduce the number of attacked snippets, we extend the optimization objective in Equation 3, which targets a single query, by aggregating the influence and query-similarity terms over a set of queries. A single adversarial snippet can therefore be optimized against several queries at once. Using this multi-query objective, we generate snippets that each target 2 or 4 queries, producing the "10 attacked snippets" and "5 attacked snippets" settings, respectively.

Table 4 reports retrieval performance under the shared-corpus attack. Injecting 20 adversarial snippets, i.e., 4% of the corpus, sharply reduces MRR and NDCG for all Eval Models. Recall@5 is largely unaffected: with one adversarial snippet per

Table 4: Retrieval performance under the shared-corpus attack across different numbers of injected snippets. Values in parentheses denote the absolute drop (↓) relative to the unattacked baseline; (→) indicates no change.

Model	MRR	NDCG	Recall@5
20 attacked snippets			
CodeT5+	40.8 (↓ 38.2)	54.6 (↓ 29.6)	90.0 (↓ 5.0)
OASIS	67.0 (↓ 11.9)	74.9 (↓ 9.0)	90.0 (→)
Nomic	66.8 (↓ 14.4)	74.0 (↓ 10.8)	95.0 (→)
10 attacked snippets			
CodeT5+	69.8 (↓ 9.3)	77.2 (↓ 7.0)	90.0 (↓ 5.0)
OASIS	73.9 (↓ 5.0)	80.1 (↓ 3.8)	90.0 (→)
Nomic	76.2 (↓ 5.1)	81.0 (↓ 3.8)	95.0 (→)
5 attacked snippets			
CodeT5+	74.7 (↓ 4.4)	80.8 (↓ 3.4)	90.0 (↓ 5.0)
OASIS	74.5 (↓ 4.3)	80.5 (↓ 3.4)	90.0 (→)
Nomic	75.4 (↓ 5.8)	80.4 (↓ 4.4)	95.0 (→)

Table 5: Impact of adversarial attacks on LLMs evaluated on RepoQA. All reported statistics are accuracy percentages. We compare accuracy in the Original setting against the Full Attack setting and its ablations: w/o Grad. removes gradient information, w/o Style removes style preservation, and w/o QTS removes query-token similarity. The results reveal a severe reduction in accuracy for both `GPT-5.4-mini` and `Gemini-3.1-Pro`, confirming their susceptibility to our attack.

Model	Setting	Python	C++	Java	Rust	TypeScript
GPT-5.4-mini	Original	96	92	94	92	99
	Full Attack	81	70	73	78	83
	w/o Grad.	87	79	81	86	86
	w/o Style	93	87	88	90	94
	w/o QTS	89	85	85	90	93
Gemini-3.1-Pro	Original	95	92	95	95	98
	Full Attack	79	72	77	80	82
	w/o Grad.	88	80	79	88	85
	w/o Style	91	90	89	92	95
	w/o QTS	87	81	82	90	93

query, the ground truth is displaced from the top rank but rarely beyond the top five. The 10- and 5-snippet settings confirm that the multi-query objective remains effective, degrading MRR and NDCG with only 1-2% of the corpus attacked, though the effect weakens as each snippet is shared among more queries, revealing a trade-off between the number of attacked snippets and per-query attack strength.

4.5 Effectiveness on LLMs

We further test whether the attack transfers from embedding-based code search to LLM-based repository retrieval. Following RepoQA (Liu et al., 2024a), each LLM receives a detailed natural-language query and a candidate set, then selects the matching function; accuracy is the evaluation metric. For each query, we replace only **two** irrelevant candidates (less than 10% of the input tokens) with adversarial snippets generated using `CodeT5+`.

Table 6: Retrieval and robustness trade-off after finetuning. PT-CodeT5+ is generated on pretrained `CodeT5+`; Trans-OASIS is transferred from `OASIS`; FT-CodeT5+ is generated directly on the evaluated finetuned `CodeT5+` checkpoint.

Checkpoint	Retrieval			Robustness (Δ Sim.)		
	MRR	NDCG	R@5	PT-CodeT5+ Attack	OASIS Transfer	FT-CodeT5+ Attack
Pretrain Base	72.4	77.3	88.0	39.90	22.25	–
Robust-Only FT	34.1	37.6	42.6	-22.70	-15.70	7.14
Mixed FT	72.3	77.6	89.4	4.00	4.42	34.70

As shown in Table 5, this small perturbation can transfer to LLMs and consistently reduce accuracy for both `GPT-5.4-mini` and `Gemini-3.1-Pro` across Python, C++, and Java. The ablation rows indicate that each component contributes to the LLM transfer effect; we discuss these components in more detail in Section 4.7.

4.6 Defense via Robust Finetuning

We also examine whether finetuning can defend against our attack.[2] As summarized in Table 6, the first key finding is that Robust-Only FT can mitigate the attack's influence, driving static attack-induced Δ Sim. below zero and substantially reducing the white-box attack effect. However, this robustness comes with a prohibitive retrieval cost: the main retrieval metrics fall to roughly half of the pretrained baseline, with R@5 dropping from 88.0 to 42.6. The second key finding is that Mixed FT appears to balance retrieval quality and static robustness better, preserving baseline search performance while reducing the impact of static attacks. Nevertheless, this balance breaks under an adaptive white-box threat model: when the attack is generated directly on the finetuned checkpoint, Mixed FT remains highly vulnerable. Moreover, although not shown in the main table, Appendix E.9 shows that attacks on the Mixed FT checkpoint also transfer to other Eval Models. Overall, standard adversarial finetuning is therefore insufficient as a defense: robust-only training sacrifices retrieval utility, while mixed training preserves utility but leaves an effective adaptive and transferable attack surface.

[2] Details of Robust-Only FT and Mixed FT are provided in Appendix E.9.

Table 7: Ablation study quantifying the contribution of specific components. Improved Counts denote the number of pairs with increased similarity after the attack.

Method	Eval Model	Δ Sim.	Improved Counts
Full Method	CodeT5+	38.86 ± 10.44	9,899
	Nomic	39.25 ± 10.89	9,895
w/o gradient	CodeT5+	30.91 ± 12.42	9,855
	Nomic	35.35 ± 12.66	9,848
w/o style preservation	CodeT5+	39.48 ± 10.39	9,899
	Nomic	39.95 ± 10.86	9,895
w/o query token sim.	CodeT5+	30.93 ± 10.08	9,893
	Nomic	22.68 ± 11.01	9,766

4.7 Ablation Studies

To isolate the contribution of each component described in Sections 3.2, 3.3, and 3.4, we remove each component from the full method and evaluate both embedding-level similarity on 10,000 CosQA pairs (Table 7) and downstream LLM retrieval accuracy on RepoQA (Table 5). In the experiment, CodeT5+ is still used as the surrogate model. Other ablation studies on the selection of the adversarial code snippet, the number of iterations, and the choice of the hyperparameter α are provided in Appendix E.10.

Without gradient guidance, the attack relies solely on query similarity, often replacing identifiers with query tokens by default. When style constraints restrict the search, it selects the most similar style-compliant candidate instead. As shown in Table 7, removing gradient information reduces Δ Sim. for both `CodeT5+` and `Nomic-embed-code`, and the larger standard deviations indicate less stable similarity changes. Since backpropagation consumes most of the computational cost, the gradient-free variant can still serve as a lightweight alternative when resources are limited, but it is consistently weaker than the full method.

Removing style preservation yields negligible, and occasionally positive, changes in raw embedding similarity. This suggests that when an optimal token violates style rules, a style-compliant substitute with comparable embedding efficacy often exists. Further analysis reveals distinct behaviors regarding underscores between evaluation models: in `CodeT5+`, "_" is typically parsed as an independent token, so replacing it has little influence; in `OASIS`, underscores are usually treated as token prefixes, and in approximately 90% of cases, a top-3 candidate token also begins with an underscore. However, Table 5 shows that style preservation is crucial for generative models. Unconstrained attacks may preserve embedding similarity, but they produce visibly incoherent identifiers that are less convincing to LLMs, causing `GPT-5.4-mini` and `Gemini-3.1-Pro` to recover much of their original RepoQA accuracy.

Query token similarity is also essential for transferability. Removing this component causes a significant drop in embedding effectiveness, particularly on `Nomic-embed-code`. Further analysis indicates that in 83% of cases, the optimal replacement tokens derived from the full method (Equation 3) are among the top-20 candidates with the highest influence. Table 5 further shows that discarding query token similarity substantially weakens the attack against LLMs. By prioritizing similarity to query tokens, the full method inserts semantically relevant vocabulary rather than only mathematically optimized gradient tokens, improving the naturalness and coherence needed to mislead generative models.

5 Conclusion & Future Work

We present a transferable adversarial attack that modifies code identifiers to mislead code search models. The attack is effective across embedding models, retrieval benchmarks, and LLM-based repository retrieval, revealing that current CLMs still rely heavily on lexical cues rather than robust semantic understanding.

Future work should investigate why attacks transfer across models and develop defenses that preserve retrieval utility. Promising directions include functionality-aware contrastive training and incorporating programming-language structure, such as ASTs, to build more semantics-grounded code representations.

Acknowledgements

We thank the anonymous reviewers, ACs, and PCs for their constructive feedback. We also thank Jiaqi Lu for insightful discussions during the early stages of this work. This research was supported in part by the U.S. National Science Foundation (NSF) under grants 2409005 and 2321444. Any opinions, findings, and conclusions in this paper are those of the authors only and do not necessarily reflect the views of our sponsors.

References

Junkai Chen, Xing Hu, Zhenhao Li, Cuiyun Gao, Xin Xia, and David Lo. 2024. Code search is all you need? improving code suggestions with code search. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ICSE '24, New York, NY, USA. Association for Computing Machinery.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, and 39 others. 2021. Evaluating large language models trained on code. *Preprint*, arXiv:2107.03374.

Yuchen Chen, Weisong Sun, Chunrong Fang, Zhenpeng Chen, Yifei Ge, Tingxu Han, Quanjun Zhang, Yang Liu, Zhenyu Chen, and Baowen Xu. 2025. Security of language models for code: A systematic literature review. *ACM Trans. Softw. Eng. Methodol.* Just Accepted.

Luca Di Grazia and Michael Pradel. 2023. Code search: A survey of techniques for finding code. *ACM Comput. Surv.*, 55(11).

Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. Codebert: A pre-trained model for programming and natural languages. *Preprint*, arXiv:2002.08155.

Zuchen Gao, Zizheng Zhan, Xianming Li, Erxin Yu, Ziqi Zhan, Haotian Zhang, Bin Chen, Yuqun Zhang, and Jing Li. 2025. Oasis: Order-augmented strategy for improved code search. *Preprint*, arXiv:2503.08161.

Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. 2015. Explaining and harnessing adversarial examples. *Preprint*, arXiv:1412.6572.

Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. 2022. UniXcoder: Unified cross-modal pre-training for code representation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7212–7225, Dublin, Ireland. Association for Computational Linguistics.

Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, Michele Tufano, Shao Kun Deng, Colin Clement, Dawn Drain, Neel Sundaresan, Jian Yin, Daxin Jiang, and Ming Zhou. 2021. Graphcodebert: Pre-training code representations with data flow. *Preprint*, arXiv:2009.08366.

Max Hort, Linas Vidziunas, and Leon Moonen. 2025. Semantic-preserving transformations as mutation operators: A study on their effectiveness in defect detection. In *2025 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*, pages 337–346.

Al Amin Hossain, Mithun Kumar PK, Junjie Zhang, and Fathi Amsaad. 2024. Malicious code detection using llm. In *NAECON 2024 - IEEE National Aerospace and Electronics Conference*, pages 414–416.

Kristen Howell, Gwen Christian, Pavel Fomitchov, Gitit Kehat, Julianne Marzulla, Leanne Rolston, Jadin Tredup, Ilana Zimmerman, Ethan Selfridge, and Joseph Bradley. 2023. The economic trade-offs of large language models: A case study. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 5: Industry Track)*, pages 248–267, Toronto, Canada. Association for Computational Linguistics.

Junjie Huang, Duyu Tang, Linjun Shou, Ming Gong, Ke Xu, Daxin Jiang, Ming Zhou, and Nan Duan. 2021. CoSQA: 20,000+ web queries for code search and question answering. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 5690–5700, Online. Association for Computational Linguistics.

Li Huang, Weifeng Sun, and Meng Yan. 2025. Iterative Generation of Adversarial Example for Deep Code Models . In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, pages 623–623, Los Alamitos, CA, USA. IEEE Computer Society.

Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, Kai Dang, and 1 others. 2024. Qwen2.5-coder technical report. *Preprint*, arXiv:2409.12186.

Paras Jain, Ajay Jain, Tianjun Zhang, Pieter Abbeel, Joseph Gonzalez, and Ion Stoica. 2021. Contrastive code representation learning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 5954–5971, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics.

Hamed Jelodar, Samita Bai, Parisa Hamedi, Hesamodin Mohammadian, Roozbeh Razavi-Far, and Ali Ghorbani. 2025. Large language model (llm) for software security: Code analysis, malware analysis, reverse engineering. *Preprint*, arXiv:2504.07137.

Akshita Jha and Chandan K. Reddy. 2023. Codeattack: code-based adversarial attacks for pre-trained programming language models. In *Proceedings of the Thirty-Seventh AAAI Conference on Artificial Intelligence and Thirty-Fifth Conference on Innovative Applications of Artificial Intelligence and Thirteenth Symposium on Educational Advances in Artificial Intelligence*, AAAI'23/IAAI'23/EAAI'23. AAAI Press.

Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2026. A survey on large language models for code generation. *ACM Trans. Softw. Eng. Methodol.*, 35(2).

Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, volume 2024, pages 54107–54157.

Harold W. Kuhn. 1955. The hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1-2):83–97.

Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, and 1 others. 2023. Starcoder: may the source be with you! *Preprint*, arXiv:2305.06161.

Xiangyang Li, Kuicai Dong, Yi Quan Lee, Wei Xia, Hao Zhang, Xinyi Dai, Yasheng Wang, and Ruiming Tang. 2024. Leaderboard of coir (code information retrieval benchmark). https://archersama.github.io/coir/. Accessed: 2025-03-31.

Xiangyang Li, Kuicai Dong, Yi Quan Lee, Wei Xia, Hao Zhang, Xinyi Dai, Yasheng Wang, and Ruiming Tang. 2025. CoIR: A comprehensive benchmark for code information retrieval models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 22074–22091, Vienna, Austria. Association for Computational Linguistics.

Chao Liu, Xin Xia, David Lo, Cuiyun Gao, Xiaohu Yang, and John Grundy. 2021. Opportunities and challenges in code search tools. *ACM Comput. Surv.*, 54(9).

Jiawei Liu, Jia Le Tian, Vijay Daita, Yuxiang Wei, Yifeng Ding, Yuhan Katherine Wang, Jun Yang, and Lingming Zhang. 2024a. Repoqa: Evaluating long context code understanding. *Preprint*, arXiv:2406.06025.

Tianyang Liu, Canwen Xu, and Julian McAuley. 2024b. Repobench: Benchmarking repository-level code auto-completion systems. In *International Conference on Learning Representations*, volume 2024, pages 47832–47850.

Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, Tianyang Liu, Max Tian, Denis Kocetkov, Arthur Zucker, Younes Belkada, Zijian Wang, Qian Liu, Dmitry Abulkhanov, Indraneil Paul, and 47 others. 2024. Starcoder 2 and the stack v2: The next generation. *Preprint*, arXiv:2402.19173.

CheolWon Na, YunSeok Choi, and Jee-Hyong Lee. 2023. DIP: Dead code insertion based black-box attack for programming language model. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7777–7791, Toronto, Canada. Association for Computational Linguistics.

Thanh-Dat Nguyen, Yang Zhou, Xuan Bach D. Le, Patanamon Thongtanunam, and David Lo. 2024. Adversarial attacks on code models with discriminative graph patterns. *Preprint*, arXiv:2308.11161.

Liming Nie, He Jiang, Zhilei Ren, Zeyi Sun, and Xiaochen Li. 2016. Query expansion based on crowd knowledge for code search. *IEEE Transactions on Services Computing*, 9(5):771–783.

Nomic Team. 2025. Nomic Embed Code: A State-of-the-Art Code Retriever. https://www.nomic.ai/news/introducing-state-of-the-art-nomic-embed-code. Nomic News, March 27, 2025. Accessed May 5, 2025.

OpenAI. 2024. New embedding models and api updates.

Simon Ott, Adriano Barbosa-Silva, Kathrin Blagec, Jan Brauner, and Matthias Samwald. 2022. Mapping global dynamics of benchmark creation and saturation in artificial intelligence. *Nature Communications*, 13(1):6793.

Rachel Potvin and Josh Levenberg. 2016. Why google stores billions of lines of code in a single repository. *Communications of the ACM*, 59(7):78–87.

Yubin Qu, Song Huang, and Yongming Yao. 2024. A survey on robustness attacks for deep code models. *Automated Software Engg.*, 31(2).

Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, and 7 others. 2024. Code llama: Open foundation models for code. *Preprint*, arXiv:2308.12950.

Gaurav Shekhar. 2024. The impact of ai and automation on software development: A deep dive. https://ieeechicago.org/the-impact-of-ai-and-automation-on-software-development-a-deep-dive/. IEEE Chicago Section. Accessed 2025-05-01.

Weisong Sun, Chunrong Fang, Yifei Ge, Yuling Hu, Yuchen Chen, Quanjun Zhang, Xiuting Ge, Yang Liu, and Zhenyu Chen. 2024. A survey of source code search: A 3-dimensional perspective. *ACM Trans. Softw. Eng. Methodol.*, 33(6).

Voyage AI. 2024. voyage-code-3: more accurate code retrieval with lower dimensional, quantized embeddings. Voyage AI blog post.

Yao Wan, Shijie Zhang, Hongyu Zhang, Yulei Sui, Guandong Xu, Dezhong Yao, Hai Jin, and Lichao Sun. 2022. You see what i want you to see: poisoning vulnerabilities in neural code search. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2022, page 1233–1245, New York, NY, USA. Association for Computing Machinery.

Kaicheng Wang, Liyan Huang, Weike Fang, and Weihang Wang. 2026. Clarc: C/c++ benchmark for robust code search. In *The Fourteenth International Conference on Learning Representations*.

Xin Wang, Yasheng Wang, Fei Mi, Pingyi Zhou, Yao Wan, Xiao Liu, Li Li, Hao Wu, Jin Liu, and Xin Jiang. 2021a. Syncobert: Syntax-guided multi-modal contrastive pre-training for code representation. *Preprint*, arXiv:2108.04556.

Yuchen Wang, Shangxin Guo, and Chee Wei Tan. 2025a. From code generation to software testing: Ai copilot with context-based retrieval-augmented generation. *IEEE Software*, 42(4):34–42.

Yue Wang, Hung Le, Akhilesh Gotmare, Nghi Bui, Junnan Li, and Steven Hoi. 2023. CodeT5+: Open code large language models for code understanding and generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 1069–1088, Singapore. Association for Computational Linguistics.

Yue Wang, Weishi Wang, Shafiq Joty, and Steven C. H. Hoi. 2021b. Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. *Preprint*, arXiv:2109.00859.

Zora Zhiruo Wang, Akari Asai, Xinyan Velocity Yu, Frank F. Xu, Yiqing Xie, Graham Neubig, and Daniel Fried. 2025b. CodeRAG-bench: Can retrieval augment code generation? In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 3199–3214, Albuquerque, New Mexico. Association for Computational Linguistics.

Jin Wen, Qiang Hu, Yuejun Guo, Maxime Cordy, and Yves Le Traon. 2025. Variable renaming-based adversarial test generation for code model: Benchmark and enhancement. *ACM Trans. Softw. Eng. Methodol.* Just Accepted.

Chen Wu and Ming Yan. 2022. Learning deep semantic model for code search using codesearchnet corpus. *Preprint*, arXiv:2201.11313.

Di Wu, Wasi Uddin Ahmad, Dejiao Zhang, Murali Krishna Ramanathan, and Xiaofei Ma. 2024. Repoformer: Selective retrieval for repository-level code completion. *Preprint*, arXiv:2403.10059.

An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma, and 43 others. 2024. Qwen2 technical report. *Preprint*, arXiv:2407.10671.

Zhou Yang, Jieke Shi, Junda He, and David Lo. 2022. Natural attack for pre-trained models of code. In *Proceedings of the 44th International Conference on Software Engineering*, ICSE '22, page 1482–1493, New York, NY, USA. Association for Computing Machinery.

Kaichun Yao, Hao Wang, Chuan Qin, Hengshu Zhu, Yanjun Wu, and Libo Zhang. 2024. Carl: Unsupervised code-based adversarial attacks for programming language models via reinforcement learning. *ACM Trans. Softw. Eng. Methodol.*, 34(1).

Noam Yefet, Uri Alon, and Eran Yahav. 2020. Adversarial examples for models of code. *Proc. ACM Program. Lang.*, 4(OOPSLA).

Huangzhao Zhang, Zhuo Li, Ge Li, Lei Ma, Yang Liu, and Zhi Jin. 2020a. Generating adversarial examples for holding robustness of source code processing models. *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(01):1169–1176.

Huangzhao Zhang, Shuai Lu, Zhuo Li, Zhi Jin, Lei Ma, Yang Liu, and Ge Li. 2024. Codebert-attack: Adversarial attack against source code deep learning models via pre-trained model. *J. Softw. Evol. Process*, 36(3).

Wei Emma Zhang, Quan Z. Sheng, Ahoud Alhazmi, and Chenliang Li. 2020b. Adversarial attacks on deep-learning models in natural language processing: A survey. *ACM Trans. Intell. Syst. Technol.*, 11(3).

Penghao Zhao, Hailin Zhang, Qinhan Yu, Zhengren Wang, Yunteng Geng, Fangcheng Fu, Ling Yang, Wentao Zhang, Jie Jiang, and Bin Cui. 2024. Retrieval-augmented generation for ai-generated content: A survey. *Preprint*, arXiv:2402.19473.

Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shan Wang, Yufei Xue, Lei Shen, Zihan Wang, Andi Wang, Yang Li, Teng Su, Zhilin Yang, and Jie Tang. 2023. Codegeex: A pre-trained model for code generation with multilingual benchmarking on humaneval-x. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD '23, page 5673–5684, New York, NY, USA. Association for Computing Machinery.

Yu Zhou, Xiaoqing Zhang, Juanjuan Shen, Tingting Han, Taolue Chen, and Harald Gall. 2022. Adversarial robustness of deep code comment generation. *ACM Trans. Softw. Eng. Methodol.*, 31(4).

A Limitations

Although we demonstrate that adversarial attacks can transfer across various code embedding models, the underlying causes of this transferability remain unclear. One possible explanation is that these models share overlapping pretraining data. However, because only a limited number of evaluated models have publicly available pretraining details, we cannot draw a definitive conclusion.

B Reproducibility

The experiments described in this paper were conducted on a server equipped with an AMD EPYC Milan 7643 48-core CPU (@2.30GHz), 1TB of RAM, and an NVIDIA L40 Ada 48GB GPU. We used a batch size of 10 *(query, code)* pairs for attacks on `CodeT5+` and 4 pairs for attacks on `OASIS`. We observed that larger batch sizes did not substantially change the runtime. The time and GPU memory required for the experiments are reported in Table 8.

To mitigate the risk of potential misuse, we do not publicly release the attack codebase. However, researchers interested in reproducing or extending these results are encouraged to contact the first author to discuss our implementation details.

C Model & Dataset Details

Models. Table 9 summarizes the models used in our experiments. `OASIS`, `Nomic-embed-code`, and `Voyage-code-3` use highly similar modified Qwen2 tokenizers (Yang et al., 2024): `OASIS` and `Voyage-code-3` share the same tokenizer, while `Nomic-embed-code` differs only in the index assigned to the `<EOS>` token. For closed-source models, parameter counts and identifier-token counts are unavailable.

Datasets. Table 10 summarizes the datasets used in the experiments.

C.1 Model & Dataset License

- **CodeT5+**: BSD 3-Clause License [3]
- **OASIS**: MIT License[4]
- **Nomic-embed-code**: Apache-2.0 [5]
- **Voyage-code-3**: The license is unclear; however, we do not include embeddings from `Voyage-code-3` in our codebase.
- **CosQA**: Apache-2.0[6] (we use CosQA from COIR)
- **CLARC**: CC-BY-SA 4.0[7]
- **RepoQA**: Apache-2.0[8]
- **HumanEval-X**: Apache-2.0[9]

D Implementation Details

D.1 Detailed Constraints

In our adversarial attack, we focus on replacing identifier tokens—specifically, tokens that comprise function, variable, macro, and module names in the code text. Formally, let the original code text be tokenized as $\{C_{t_i}\}_{i=1}^{n}$, and the code text after replacement be tokenized as $\{C'_{t_i}\}_{i=1}^{n}$. For any two strings A and B, let $\mathrm{LCS}(A, B)$ denote their longest common substring, and let $\mathrm{Remove}(A, B)$ denote the remaining string after removing string B from string A.

The primary challenge in replacing these identifiers is accounting for tokenization corner cases, where varying punctuation or whitespace is merged with the identifier itself. To address this, we introduce two additional constraints during replacement. We illustrate the intuition behind these constraints using the following original code snippet (top) and its adversarially modified version (bottom):

```python
# Original snippet
def print_runs(query):
    if query is None:
        return
    for tup in query:
        print(("{0} @ {1} - {2} id: {3} group: {4}".format(
            tup.end, tup.experiment_name, tup.project_name,
            tup.experiment_group, tup.run_group)))
```

```python
# Modified snippet
def off_runs(number):
    if number is None:
        return
    for to in number:
        print(("{0} @ {1} - {2} id: {3} group: {4}".format(
            to.end, to.experiment_name, to.project_name,
            to.experiment_group, to.run_group)))
```

Identifier Consistency. This constraint ensures that all occurrences of the same variable or func-

[3] https://github.com/salesforce/CodeT5?tab=BSD-3-Clause-1-ov-file

[4] https://huggingface.co/Kwaipilot/OASIS-code-embedding-1.5B

[5] https://huggingface.co/nomic-ai/nomic-embed-code

[6] https://github.com/CoIR-team/coir/blob/main/LICENSE

[7] https://huggingface.co/datasets/ClarcTeam/CLARC

[8] https://github.com/evalplus/repoqa/blob/main/LICENSE

[9] https://huggingface.co/datasets/THUDM/humaneval-x

Table 8: Compute resources used to generate 10,000 adversarial examples.

Surrogate Model	Total Time	GPU Time	CPU Time	GPU Memory	Token Search Space
CodeT5+ (110M)	8 hours	∼6.8 hours (85%)	∼1.2 hours (15%)	7GB	15.1k
OASIS (1.5B)	12 hours	∼10.6 hours (88%)	∼1.4 hours (12%)	26GB	36.7k

Table 9: Models used in the experiments.

Model	# Parameters	Vocabulary Size	# Tokens valid for identifiers
CodeT5+	110M	32,103	29,881
OASIS	1.54B	151,665	74,194
Nomic-embed-code	7.07B	151,665	74,194
Voyage-code-3	-	151,665	74,194
GPT-5.4-mini	-	-	-
Gemini-3.1-Pro	-	-	-

tion name are replaced consistently, even when tokenizer artifacts attach different characters to the identifier. If C_{t_i} and C_{t_j} are tokens from different occurrences of the same identifier, let $L_{ij} = \text{LCS}(C_{t_i}, C_{t_j})$ and $L'_{ij} = \text{LCS}(C'_{t_i}, C'_{t_j})$. The constraint is then:

$$\text{Remove}(C'_{t_i}, L'_{ij}) = \text{Remove}(C_{t_i}, L_{ij}),$$
$$\text{Remove}(C'_{t_j}, L'_{ij}) = \text{Remove}(C_{t_j}, L_{ij}).$$

For instance, the OASIS tokenizer includes different surrounding characters in the tokens corresponding to the two occurrences of the identifier `query`. The original and modified tokens are:

- On line 2, let $C_{t_i} =$ `(query`. After replacing `query` with `number`, the corresponding token becomes $C'_{t_i} =$ `(number`.
- On line 5, let $C_{t_j} =$ `␣query`, where `␣` denotes the leading whitespace. After replacement, the token becomes $C'_{t_j} =$ `␣number`.

Although the complete tokens differ because of their surrounding characters, their longest common substrings isolate the identifier itself: $L_{ij} =$ `query` before the attack and $L'_{ij} =$ `number` afterward. Removing these identifier substrings leaves the surrounding token context unchanged:

- For line 2, $\text{Remove}(C'_{t_i}, L'_{ij}) = \text{Remove}(C_{t_i}, L_{ij}) =$ `(`.
- For line 5, $\text{Remove}(C'_{t_j}, L'_{ij}) = \text{Remove}(C_{t_j}, L_{ij}) =$ `␣`.

Thus, the constraint permits the identifier replacement while preserving the parenthesis and whitespace attached by the tokenizer.

No Duplicate Replacement. This constraint guarantees that distinct variables or functions are replaced by distinct new identifiers. If C_{t_i}, C_{t_j} are tokens from occurrences of one identifier, and C_{t_p}, C_{t_q} are tokens from occurrences of a *different* identifier, then:

$$\text{LCS}(C'_{t_i}, C'_{t_j}) \neq \text{LCS}(C'_{t_p}, C'_{t_q}).$$

In the example, the tokens are divided into two groups according to the original identifier from which they are derived:

- For the original identifier `query`, let $C_{t_i} =$ `(query` and $C'_{t_i} =$ `(number` on line 2. Similarly, let $C_{t_j} =$ `␣query` and $C'_{t_j} =$ `␣number` on line 5.
- For the distinct original identifier `tup`, let $C_{t_p} =$ `␣tup` and $C'_{t_p} =$ `␣to` on line 5. The same tokens occur again on line 7, where $C_{t_q} =$ `␣tup` and $C'_{t_q} =$ `␣to`.

The first pair therefore represents the replacement `query` → `number`, whereas the second represents `tup` → `to`. To prevent the two distinct original identifiers from being mapped to the same new identifier, the constraint enforces

$$\text{LCS}(C'_{t_i}, C'_{t_j}) \neq \text{LCS}(C'_{t_p}, C'_{t_q}),$$

Table 10: Datasets used in the experiments.

Dataset	# Queries	# Code Snippets	Programming Languages
CosQA	500	500	Python
CLARC	526	526	C++
RepoQA	200	2,232	Python, C++, Java
HumanEval-X	820	820	Python, C++, Java, JavaScript, Go

which in this example reduces to `number` $\neq$ `to`.

Together, these constraints ensure the code's AST structure remains identical. To satisfy the constraints, we employ the Hungarian Algorithm (Kuhn, 1955) to match original identifier tokens with their optimal replacements.

Let V denote the tokenizer's vocabulary and $S \subseteq V$ be the set of distinct tokens in the identifiers in the code snippet. For each original token $s_i \in S$, we define an influence function $f_{s_i} : V \to \mathbb{R}$ derived from gradient information. This function, $f_{s_i}(v)$, quantifies the *influence* when s_i is replaced by v.

The goal is to find a set of matching $\{s_i, v_i\}$ that maximizes the total influence, subject to the constraint that each substitute token v_i must be unique. This can be formulated as the following optimization problem:

$$\begin{aligned} \underset{\{v_i\}}{\text{maximize}} \quad & \sum_i f_{s_i}(v_i), \\ \text{subject to} \quad & v_i \in V, \\ & v_i \neq v_j \quad \forall i \neq j. \end{aligned} \tag{4}$$

D.2 Search Space

For each token position, we may need to consider up to $|\mathcal{V}|$ candidate tokens to select the replacement token that maximizes the objective in Eq. 3 (i.e., the combined influence and query-similarity score defined in the methodology). Here, we follow the notation in Section 3.2: $\mathcal{V}$ denotes the model's full tokenizer vocabulary, and $V \subseteq \mathcal{V}$ denotes the subset of identifier-compatible tokens that satisfies the style preservation requirement used for replacement. In the worst case, $|V|$ can be as large as $|\mathcal{V}|$, but in practice identifier constraints reduce the search space substantially; typically, only about 30–40% of the full vocabulary is considered.

D.3 Specification

The first step of our adversarial attack is to parse the code snippet and locate identifiers suitable for modification. We use language-specific AST parsers: Python's built-in `ast` library, `clang` for C++, and `tree-sitter` for Java, JavaScript, and Go. After applying adversarial modifications, we re-run the parsers to ensure the modified code snippet remains syntactically valid.

For the surrogate models, a replacement token is considered valid only if it consists of characters permitted in standard identifiers across programming languages. However, OASIS presents a unique challenge due to its significantly larger vocabulary: tokens representing an identifier may include preceding operators. For example, a variable `x` might be tokenized as "␣`x`" (a **single** token containing the leading white space) and "`+x`" (another **single** token containing the leading plus sign) at two occurrences in the code snippet. When searching for a replacement variable `y`, we strictly filter for candidate tokens that preserve this structure—"␣`y`" and "`+y`", respectively—and select the one that maximizes influence. Consequently, the search space for OASIS extends to tokens containing operators followed by alphanumeric characters.

E Supplementary Experimental Results

E.1 "Embedding Density" across Models

In this subsection, we use the term *embedding density* informally to describe how concentrated query–code similarity scores are in the embedding space. We quantify this using the *gap* between the average similarity for query–ground-truth pairs and for query–irrelevant pairs: a smaller gap indicates a "denser" similarity distribution (i.e., weaker separation between relevant and irrelevant snippets).

Table 11 reports these averages and gaps. Overall, CosQA tends to exhibit a more "dense" similarity distribution for some models, largely because irrelevant snippets can still attain relatively high similarity to the query. This is most visible for `OASIS`, where the gap is comparatively small (e.g., 21.40 on CosQA). Such compressed separation is consistent with our observation in the main paper that attacks evaluated on `OASIS` tend to yield smaller Δ Sim than on `CodeT5+`.

Table 11: Average similarity scores and query–code separation gaps for different models on CosQA and CLARC.

Dataset	Model	Avg. Sim between Query and GroundTruth Code	Avg. Sim between Query and Irrelevant Code	Gap
CosQA	CodeT5+	54.15	20.60	33.55
	OASIS	68.53	47.13	21.40
	Nomic-embed-code	41.77	1.95	39.82
	Voyage-code-3	71.43	37.83	33.60
CLARC	CodeT5+	52.91	25.80	27.11
	OASIS	85.80	54.61	31.19
	Nomic-embed-code	55.71	7.44	48.27
	Voyage-code-3	73.93	40.08	33.85

In contrast, CLARC generally shows larger gaps (e.g., 31.19 for `OASIS`), suggesting a larger "semantic safety margin" between ground-truth and irrelevant snippets. This aligns with the weaker overall attack impact on CLARC reported in the paper: adversarially modified irrelevant code must bridge a larger baseline separation to displace the ground truth.

E.2 Details about Metrics

Correlation Metrics

- **Precision**: The ratio that an attack induced a positive Δ Similarity on the Surrogate Model also successfully increases similarity on the Eval Model.
- **Pearson Correlation Coefficient (r)**: Measures the linear correlation between the numerical values of the similarity changes observed on the Attack and Eval Models.
- **Spearman's Rank Correlation Coefficient (ρ)**: Measures the monotonic correlation, assessing how well the rank order of similarity changes is preserved between the Attack and Eval Models.

Retrieval Metrics

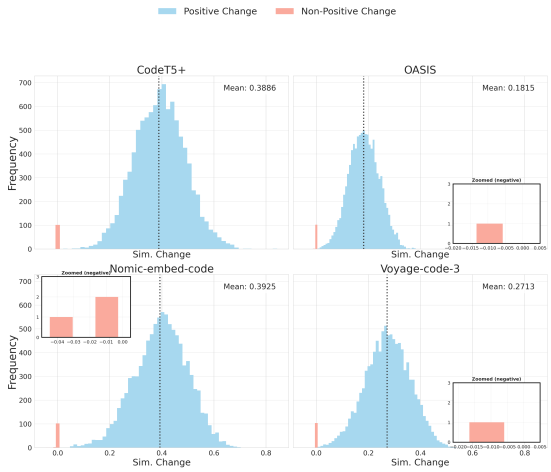
- **Recall@k (R@k)**: The proportion of queries for which the correct code snippet is found within the top-k ranked results returned by the model. Since each query in the CosQA and CLARC datasets has exactly one ground-truth matching code snippet, R@k specifically measures the percentage of queries where this single correct snippet appears among the top k candidates.
- **Normalized Discounted Cumulative Gain (NDCG)**: A metric for evaluating the quality of a ranked list. It assigns higher scores when relevant items are placed higher in the ranking, applying a logarithmic discount based on position. The score is normalized against the ideal ranking, resulting in a value between 0 and 1.
- **Mean Reciprocal Rank (MRR)**: The average of the reciprocal ranks across all queries in the test set. For a single query, the reciprocal rank is the inverse of the rank position (1/rank) of the ground truth code snippet.

E.3 Distribution of Code Similarity Changes

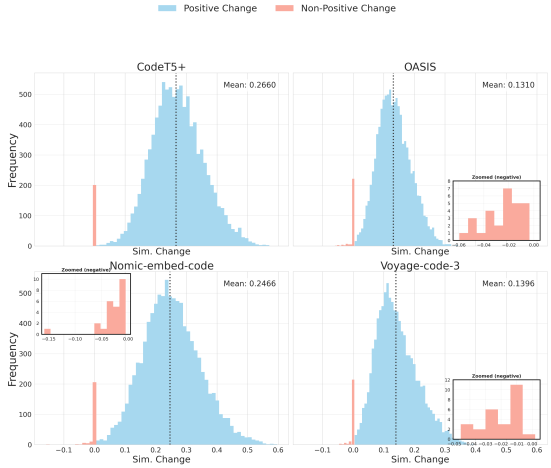
Figure 3 illustrates the distribution of cosine similarity changes (ΔSim) induced by the adversarial code snippets on the CosQA and CLARC datasets. We visualize the results across four distinct embedding models: CodeT5+, OASIS, Nomic-embed-code, and Voyage-code-3.

Prevalence of Positive Shifts. The histograms demonstrate that the vast majority of adversarial perturbations result in a positive increase in similarity scores (shown in blue). As indicated by the zoomed-in parts, instances of non-positive change (shown in red), where the adversarial code failed to increase similarity, are almost negligible. The trend confirms the attack's high transferability and consistency across different model architectures.

Dataset Comparison. Comparing the two datasets, we observe that the magnitude of Δ Sim. is consistently larger in CosQA (Python) than in CLARC (C++). For example, the mean ΔSim on CosQA ranges from approximately 0.18 to 0.39, whereas on CLARC, the means are more constrained, ranging from 0.13 to 0.27. This suggests that the generated adversarial perturbations are generally more effective at inducing large shifts in the embedding space for Python code snippets compared to C++.



(a) CosQA



(b) CLARC

Figure 3: Distribution of code similarity changes. The surrogate model is `CodeT5+`, and the eval models are labeled at the top of each subplot.

Model Sensitivity. There is also a notable variance in model sensitivity. `Nomic-embed-code` and `CodeT5+` consistently exhibit the highest mean similarity shifts. In contrast, `OASIS` and `Voyage-code-3` show more conservative shifts. The magnitude of similarity shift can also be explained by *embedding density* from Appendix E.1.

E.4 Query Transferability

In practice, developers rarely use identical phrasing for the same semantic intent, meaning a robust attack must generalize across lexical variations. To evaluate this, we use GPT-5.4-mini to generate three paraphrases for each original query in our dataset. We manually validate that the exact intent is preserved. The paraphrased queries introduce a substantial ratio of new tokens not present in the originals: 41.39% for CosQA and 34.81% for CLARC. Crucially, the attacked code snippets are optimized against the original queries. We then compare the Δ Sim. with respect to the original queries and the Δ Sim. with respect to the corresponding paraphrases, which the optimization never observes.

Table 12: Δ Sim. caused by attacked code snippets when evaluated against original queries (Ori. Q) versus paraphrased queries (Para. Q). The closely matching Δ Sim. values across both query types demonstrate that our adversarial attack transfers effectively to queries sharing the same semantic intent.

Dataset	Eval Model	Δ Sim. Ori. Q	Δ Sim. Para. Q
CosQA	CodeT5+	38.86 ± 10.44	33.33 ± 12.03
	OASIS	18.15 ± 6.03	17.14 ± 6.22
	Nomic	39.25 ± 10.89	34.96 ± 12.04
	Voyage-code-3	27.13 ± 8.96	25.23 ± 9.24
CLARC	CodeT5+	11.20 ± 8.66	10.58 ± 8.81
	OASIS	13.54 ± 5.46	13.00 ± 5.38
	Nomic	17.78 ± 8.36	17.35 ± 8.27
	Voyage-code-3	11.81 ± 6.42	12.49 ± 6.69

Table 13: Cross-model transferability of baseline adversarial attacks. The table details the similarity shifts (Δ Sim.) and Spearman correlation coefficients (ρ) when adversarial snippets generated on a source attack model are evaluated across different target models.

Method	Attack Model	Eval Model	Δ Sim.	ρ
DAMP	OASIS	OASIS	10.5 ± 5.5	–
		CodeT5+	10.8 ± 11.8	63.8
		Nomic	16.2 ± 11.0	79.6
		Voyage	12.7 ± 8.4	84.1
CodeAttack	Voyage	Voyage	5.2 ± 4.2	–
		CodeT5+	7.1 ± 4.9	48.3
		Nomic	8.8 ± 3.7	52.7
		OASIS	5.6 ± 3.6	49.4

As the results in Table 12 indicate, the attack remains effective on the paraphrased queries. The Δ Sim. for paraphrased queries closely follows the Δ Sim. for the original queries across all evaluated models, demonstrating that our attack is targeting the underlying intent rather than fitting to specific lexical triggers. The experiment implies that an adversary does not need prior knowledge of a victim's exact search query. As long as the victim's query aligns with the target semantic intent, the adversarial code snippet will be successfully retrieved, elevating the real-world threat profile of these attacks.

E.5 Baseline Transferability

To assess the cross-model transferability of the baseline adversarial attacks (DAMP and CodeAttack), we measured the similarity shifts (Δ Sim.) induced by their modified code snippets when evaluated on target models. We also report the Spearman correlation coefficients (ρ) for these transfers, as summarized in Table 13.

The results confirm that the baseline attacks exhibit transferability across different models. Nevertheless, comparing Table 13 with Table 1 shows that our method achieves higher similarity changes and stronger correlation coefficients in transfer scenarios. This comparison indicates that our method is more effective and controllable than the established baselines.

E.6 Attack on Various Programming Languages

To assess attack transfer across languages, we use the HumanEval-X dataset (Zheng et al., 2023), which includes 164 queries with solutions in Python, C++, Java, JavaScript, and Go. We create 26,896 *(query, code)* pairs by pairing each query with all non-corresponding solutions. `CodeT5+` serves as the surrogate model, while `OASIS` and `Nomic-embed-code` are used as eval models.

As shown in Table 14, the adversarial attack is effective and transferable across all five languages. On the surrogate model, all languages exhibit consistently high similarity changes and positive change counts. Meanwhile, the attack is particularly transferable against Java, JavaScript, and Go, where both `OASIS` and `Nomic-embed-code` show larger similarity increases than against Python and C++.

Despite these variations in attack magnitude, the correlation coefficients reveal a distinct pattern. Generally, the correlations are moderate across the five languages, suggesting that the monotonic relationship between the attack's impact on the surrogate model and its transferred impact on eval models is stable across languages. However, we observe that the correlation coefficients for Python are notably higher than those for the other programming languages. We hypothesize that such predictability stems from Python's dominance in pre-training corpora, which leads most code embedding models to learn more robust and more consistent representations for Python than for other languages.

E.7 Different Attack Ratio on Benchmark

We investigated the sensitivity of code retrieval models to varying adversarial attack ratios by evaluating attacks of 1%, 2%, and 5% of the corpus, alongside the original 10% baseline (Table 15). The experiments were conducted on a corpus of 500 snippets. Consequently, the 1% attack represents a strictly constrained threat model involving the manipulation of only 5 snippets.

Efficacy at low attack ratios. The results indicate that the attack remains highly potent even under minimal perturbations. Manipulating 1% of the candidate pool triggers substantial performance degradation in both white-box and black-box settings. For instance, Recall@1 drops by approximately 60% on the surrogate model (`CodeT5+`). Notably, this vulnerability transfers to the commercial embedding model `Voyage-code-3`, which suffers a 62.8% drop in Recall@1 under the same conditions. This demonstrates that code search benchmarks' robustness can be compromised by corrupting only a small fraction of the corpus.

Trend analysis and saturation. As the poisoning percentage increases from 1% to 5%, the negative impact on retrieval metrics raises, yet it exhibits a saturation effect. The degradation observed at the 5% attack rate is nearly equivalent to the drops under the 10% scenario in the main text. The plateau suggests that our method maximizes the similarity of irrelevant snippets so effectively that a small, strategic injection of adversarial examples (5% or less) is often sufficient to dominate the top-ranked results.

E.8 Application of the Adversarial Attack on Other Benchmarks

Attacking CLARC Besides CosQA, we also evaluated the impact of adversarial attacks on the CLARC dataset (Table 16). While the attack generated by the surrogate model (`CodeT5+`) still degrades performance, the magnitude of this degradation is notably smaller compared to the CosQA dataset (Table 3). For instance, while `OASIS` suffers a catastrophic drop of over 68% in Recall@1 on CosQA, it retains significantly more robustness on CLARC, with a smaller drop of roughly 25%.

This disparity in attack effectiveness stems primarily from the intrinsic properties of the datasets:

- **Larger Semantic Safety Margin:** As shown in Table 11, the initial similarity gap between the query-ground truth pairs and query-

Table 14: Effectiveness and correlation of adversarial attacks transferred from `CodeT5+` across programming languages in HumanEval-X. Δ Sim. reports the mean similarity change ± standard deviation.

Language	Eval Model	Δ Sim.	Improved Counts	Unimproved Counts	Precision	r	ρ
Python	CodeT5+	27.70 ± 9.14	26 366	530	–	–	–
	OASIS	12.41 ± 5.21	26 302	594	99.76	61.68	56.00
	Nomic-embed-code	26.47±10.08	26 290	606	99.71	62.61	55.48
C++	CodeT5+	24.40 ± 8.57	26 823	73	–	–	–
	OASIS	14.46 ± 6.04	26 667	229	99.42	51.44	49.62
	Nomic-embed-code	26.13±10.06	26 650	246	99.36	50.39	48.40
Java	CodeT5+	24.10 ± 7.38	26 865	31	–	–	–
	OASIS	15.41 ± 5.88	26 738	158	99.53	49.60	47.71
	Nomic-embed-code	28.10±10.24	26 711	185	99.43	48.12	45.77
JavaScript	CodeT5+	28.58 ± 8.14	26 866	30	–	–	–
	OASIS	16.16 ± 6.36	26 720	176	99.46	55.42	52.89
	Nomic-embed-code	30.76±10.99	26 718	178	99.45	53.07	49.83
Go	CodeT5+	27.95 ± 9.01	26 855	41	–	–	–
	OASIS	16.86 ± 6.39	26 752	144	99.62	53.97	51.52
	Nomic-embed-code	29.79±10.42	26 733	163	99.55	51.33	48.32

irrelevant pairs is generally wider in CLARC for the transfer models. For example, `OASIS` exhibits a similarity gap of 31.19 on CLARC compared to only 21.40 on CosQA. This larger initial margin creates a higher barrier for the attacker; the adversarial perturbation must bridge a significantly wider semantic distance to displace the ground truth from the top rank.

- **Attack transferability limits:** The adversarial examples generated by the surrogate attack are less transferable (in terms of Δ Sim.) on CLARC, as shown in Table 1. As a result, the adversarial code snippets often fail to reach similarity scores high enough to crowd out the ground truth during reranking.

Attacking CodeSearchNet To further demonstrate the generalizability of our approach on a large-scale corpus, we evaluated our method on CodeSearchNet (Wu and Yan, 2022). Using CodeT5+ as the surrogate model, we randomly sampled 1,000 (query, code) pairs and applied our attack. We measured the similarity improvement (Δ Sim.), the number of pairs with positive similarity changes, and Spearman's rank correlation (ρ) across multiple evaluation models.

The results in Table 17 confirm that our method remains consistently effective and highly transferable on a different corpus. Additionally, the correlation coefficients align closely with those reported in Table 1, indicating that the attack's transferability remains stable and predictable across different datasets.

E.9 Robust Finetuning

To explore potential defense mechanisms, we investigate whether adversarial finetuning can mitigate the proposed attack. Utilizing the Cross Entropy loss, we construct positive pairs from queries and their corresponding ground-truth code snippets, and employ adversarially modified snippets targeting the queries as hard negative samples for our Robust-Only finetuning (Robust-Only FT) model. To ensure the model maintains baseline retrieval capabilities, we introduce a second variant, Mixed FT, which additionally integrates batches of CosQA training data during the finetuning process.

We first evaluate the retrieval performance on CosQA and the robustness (Δ Sim.) on a test set disjoint from the training data. Here, we measure robustness against two static attacks: one crafted using the original pretrained `CodeT5+` model (Atk-P), and the other transferred from `OASIS` (Atk-O).

As reported in Table 18, Robust-Only FT exhibits strong resistance to these attacks; both Atk-P and Atk-O yield a negative Δ Sim., indicating these attacks fail to fool the Robust-Only FT checkpoint. However, the retrieval performance of this checkpoint degrades sharply, with all three retrieval metrics dropping to less than half of those achieved by the pretrained baseline. Conversely, the Mixed FT checkpoint strikes a better balance, maintaining high retrieval performance while demonstrating improved resistance to static attacks compared to the

Table 15: Model performance under different attack percentages on CosQA. Values report absolute metric drops from the unattacked baseline.

Model Name	% of Corpus Attacked	MRR Difference	NDCG Difference	R@1 Difference	R@5 Difference	R@10 Difference	R@20 Difference
CodeT5+ (Surrogate)	1%	53.54	42.28	60.00	54.00	3.20	1.00
	2%	64.99	61.27	61.60	76.60	45.80	1.80
	5%	71.28	74.28	62.60	83.40	83.00	71.00
	10%	72.29	76.06	62.80	85.80	87.40	86.40
OASIS	1%	44.29	34.80	55.80	21.80	3.80	0.40
	2%	56.98	48.33	62.40	52.60	17.80	1.00
	5%	69.09	67.77	66.00	74.80	62.00	30.00
	10%	74.58	75.87	68.40	83.40	79.00	63.60
Nomic	1%	51.26	40.09	62.00	36.00	2.80	0.40
	2%	63.35	55.67	65.80	66.80	27.60	1.20
	5%	73.51	73.36	69.00	77.60	71.80	45.00
	10%	77.07	78.42	70.80	85.00	81.80	72.80
Voyage-code-3	1%	43.91	33.79	58.80	19.40	1.20	0.40
	2%	57.08	46.84	66.40	45.00	11.60	0.60
	5%	71.34	66.97	72.80	70.80	51.20	21.40
	10%	77.54	76.63	75.00	82.00	72.60	53.20

Table 16: Retrieval metric changes on CLARC caused by adversarial attacks.

Model Name	Setting	MRR	NDCG	R@1	R@5	R@10	R@20
CodeT5+ (Surrogate)	Original	58.84	64.55	47.34	74.14	82.51	89.54
	Adversarial	2.16	3.39	0.76	3.42	7.61	16.73
	Δ	56.68	61.15	46.58	70.72	74.91	72.81
OASIS	Original	86.54	89.08	79.85	94.11	96.77	98.48
	Adversarial	65.95	71.15	54.37	81.18	87.26	92.40
	Δ	20.59	17.93	25.48	12.93	9.51	6.08
Nomic-embed-code	Original	86.23	88.61	80.04	94.11	95.82	96.96
	Adversarial	55.96	61.37	45.63	68.82	78.52	84.98
	Δ	30.28	27.25	34.41	25.29	17.30	11.98
Voyage-code-3	Original	86.92	88.98	80.99	94.30	95.06	97.53
	Adversarial	61.26	67.43	48.48	79.28	86.69	91.45
	Δ	25.66	21.54	32.51	15.02	8.37	6.08

Table 17: Attack Effectiveness and Transferability on CodeSearchNet. All attacks are using CodeT5+ as the surrogate model.

Eval Model	Δ Sim.	Pos. Change Counts	ρ
CodeT5+	42.5 ± 10.5	1,000	–
OASIS	23.7 ± 7.1	1,000	54.9
Nomic	42.2 ± 10.9	1,000	60.4
Voyage-code-3	25.9 ± 8.1	1,000	55.6

Table 18: Performance and robustness metrics of `CodeT5+` checkpoints. Finetuning with mixed data preserves retrieval baselines and mitigates static attacks. Atk-P: attack on pretrained `CodeT5+`; Atk-O: attack transferred from `OASIS`.

Checkpoint	Retrieval			Robustness (Δ Sim.)	
	MRR	NDCG	R@5	Atk-P	Atk-O
Pretrain Baseline	72.4	77.3	88.0	39.90	22.25
Robust-Only FT	34.1	37.6	42.6	-22.70	-15.70
Mixed FT	72.3	77.6	89.4	4.00	4.42

baseline.

We further evaluate white-box attacks directly on both finetuned checkpoints. In this setting, the gradients and query-token similarities are computed using the weights of the finetuned models. As reported in Table 19, the Robust-Only FT checkpoint consistently demonstrates strong robustness against white-box attacks; although the Δ Sim. is positive, the magnitude of the change remains small. In contrast, the Mixed FT checkpoint proves vulnerable to white-box attacks, yielding Δ Sim. values comparable to those in Table 1. Notably, the absolute Spearman correlation coefficients (ρ) for both checkpoints are larger than those observed in Table 1, suggesting that attack transferability from the finetuned `CodeT5+` models is more predictable.

Table 19: Effectiveness and transferability when the attack is applied directly to finetuned models.

	Robust-Only FT		Mixed FT	
Eval Model	Δ Sim.	ρ	Δ Sim.	ρ
CodeT5+ (Finetuned)	7.14	-	34.70	-
OASIS	3.87	65.78	15.54	71.30
Nomic	6.57	69.06	29.93	76.50
Voyage	5.40	70.55	22.61	73.66

Table 20: Ablation on α. All values are Δ Similarity on 1,000 sampled *(query, code)* pairs from CosQA, scaled by 100.

Model	CodeT5+	OASIS	Nomic-embed-code
$\alpha = 0.01$	33.95	14.74	28.78
$\alpha = 0.05$	39.30	18.89	40.06
$\alpha = 0.1$	39.83	19.45	42.39
$\alpha = 0.5$	38.15	19.18	42.19

Overall, our robust finetuning experiments demonstrate that standard adversarial finetuning is an insufficient mitigation strategy for our attack. The resulting checkpoints suffer from a trade-off: they either remain vulnerable to white-box attacks or achieve robustness at the cost of severely degraded retrieval performance.

E.10 Ablation Studies

We conduct additional ablation studies to evaluate the contribution of specific components within our attack method.

Attack Code Selection Our default strategy selects the adversarial code that achieves the highest similarity to the query across all iterations of the attack process. In this ablation, we compare this approach with an alternative that selects the adversarial code directly from a fixed iteration k in order to justify the necessity of our adversarial code selection strategy.

We evaluated both strategies using the 10,000 sampled *(query, code)* pairs from CosQA for 10 iterations. Figure 4 plots the average query-code similarity at each iteration k for both selection strategies under the full method and the ablation methods. The alternative approach, which selects the code at iteration k (dashed lines), leads to less effective attacks, as the average similarity plateaus and fluctuates after three iterations. In contrast, selecting the code with the maximum similarity observed up to iteration k (solid lines) allows the average similarity to increase monotonically throughout the process, achieving much higher final similarities. The comparison confirms the benefit of retaining the highest-scoring code variant across all iterations rather than only considering the final iteration's output.

Iteration Limit Our default adversarial attack protocol employs 5 iterations. As shown in Figure 4 (solid lines), the average query-code similarity continues to increase only marginally beyond this point across all evaluation models. Specifically, extending the process from 5 to 10 iterations yields a Δ Similarity increase of less than 0.02 in all cases (both surrogate and transfer settings). Given that these additional iterations double the computational cost for diminishing returns, we terminate the attack at 5 iterations. Conversely, for scenarios with strictly limited resources, the steep initial rise in the plots suggests that running the attack for just a single iteration still yields substantial similarity improvements. However, where efficiency is not a primary constraint, performing additional iterations remains beneficial for maximizing attack performance.

Choice of α We conduct an additional ablation study to determine the optimal hyperparameter α in Equation 3. In this experiment, we sample 1,000 *(query, code)* pairs from CosQA and use `CodeT5+` as the surrogate model to test $\alpha \in \{0.01, 0.05, 0.1, 0.5\}$. We then evaluate the resulting Δ Similarity on `CodeT5+`, `OASIS`, and `Nomic-embed-code`.

As reported in Table 20, `CodeT5+` exhibits comparable Δ Similarity scores when α=0.05 and α=0.1. However, regarding transferability, we observe that `OASIS` and `Nomic-embed-code` achieve higher scores with larger α values. Consequently, we selected α=0.1 for our main experiments, as it consistently yielded the highest Δ Similarity across models.

F Adversarial Attack Examples

In this section, we provide the code snippets and their target queries before and after the adversarial attack.

F.1 CosQA

Query Example `using sort to move element in to new position in list python`

Original Code Snippet

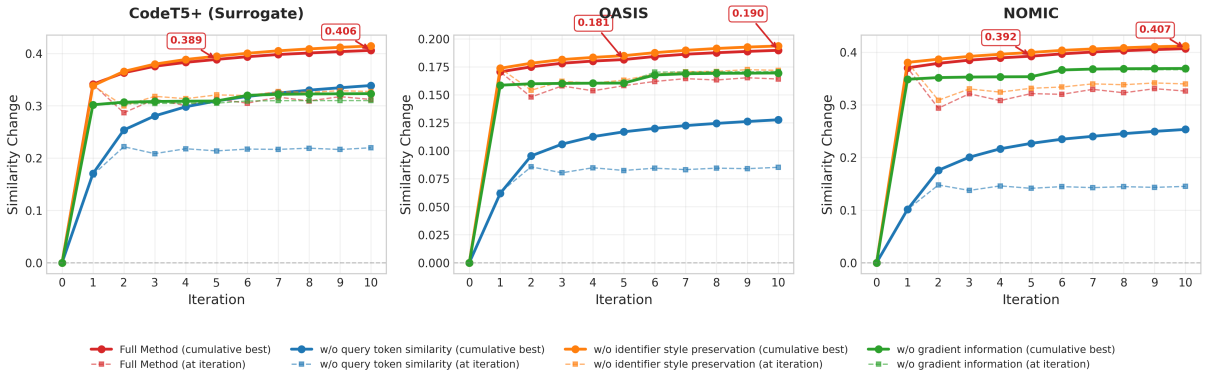


Figure 4: Average similarity change on CosQA using `CodeT5+` as the surrogate model. We evaluate the attacks on the surrogate itself (left) and two transfer targets: `OASIS` and `Nomic-embed-code` (center/right). Selecting the code with maximum similarity up to iteration k leads to higher similarity changes, while extending the attack from 5 to 10 iterations yields only marginal gains in Δ Sim.

- Similarity with Query based on CodeT5+: 0.2814
- Similarity with Query based on OASIS: 0.5312
- Similarity with Query based on Nomic-embed-code: 0.0689

```python
def process_and_sort(s, force_ascii, full_process=True):
    ts = utils.full_process(s, force_ascii=force_ascii)
    ↪ if full_process else s
    tokens = ts.split()
    sorted_string = u" ".join(sorted(tokens))
    return sorted_string.strip()
```

Attacked Code Snippet

- Similarity with Query based on CodeT5+: 0.7300 (↑ 0.4486)
- Similarity with Query based on OASIS: 0.7426 (↑ 0.2114)
- Similarity with Query based on Nomic-embed-code: 0.5925 (↑ 0.5236)

```python
def newusing_sort_move( element, force_ascii,
    ↪ full_process=True):
    ts = utils.full_process( element,
    ↪ force_ascii=force_ascii) if full_process else
    ↪ element
    move = ts.split()
    position_string = u" ".join(sorted( move))
    return position_string.strip()
```

Attacked Code Snippet (w/o gradient)

- Similarity with Query based on CodeT5+: 0.6653 (↑ 0.3839)
- Similarity with Query based on OASIS: 0.7554 (↑ 0.2242)
- Similarity with Query based on Nomic-embed-code: 0.4992 (↑ 0.4303)

```python
def sortusing_new_element( python, position_ascii,
    ↪ move_process=True):
    position = utils.full_process( python, force_ascii=
    ↪ position_ascii) if move_process else python
    tokens = position.split()
    move_string = u" ".join(sorted(tokens))
    return move_string.strip()
```

Attacked Code Snippet (w/o style preservation)

- Similarity with Query based on CodeT5+: 0.7613 (↑ 0.4799)
- Similarity with Query based on OASIS: 0.7345 (↑ 0.2033)
- Similarity with Query based on Nomic-embed-code: 0.5225 (↑ 0.4536)

```python
def sortelementsortnewusingelementsreviewList( python,
    ↪ force_ascii, new_process=True):
    to = utils.full_process( python,
    ↪ force_ascii=force_ascii) if new_process else python
    element = to.split()
    python_string = u" ".join(sorted( element))
    return python_string.strip()
```

Attacked Code Snippet (w/o query token similarity)

- Similarity with Query based on CodeT5+: 0.5808 (↑ 0.2994)
- Similarity with Query based on OASIS: 0.6336 (↑ 0.1024)
- Similarity with Query based on Nomic-embed-code: 0.3208 (↑ 0.2519)

```
def oosordering_AttachmentFileEntry_sort( Ordering,
    ↪ getTypes_Random, digest_Processes=True):
    ptr = utils.full_process( Ordering, force_ascii=
    ↪ getTypes_Random) if digest_Processes else Ordering
    duplicates = ptr.split()
    CONT_List = u" ".join(sorted( duplicates))
    return CONT_List.strip()
```

F.2 CLARC

Query Example `The function takes a constant string reference as its input and copies its content to another string reference provided as the output. It effectively performs a direct assignment, duplicating the input string without modification. The input is a constant string (std::string) and the output is also a string (std::string).`

Original Code Snippet

- Similarity with Query based on CodeT5+: 0.3851
- Similarity with Query based on OASIS: 0.6060
- Similarity with Query based on Nomic-embed-code: 0.1037

```
void fp_reverse (unsigned char *s, int len)
{
  int     ix, iy;
  unsigned char t;

  ix = 0;
  iy = len - 1;
  while (ix < iy) {
    t     = s[ix];
    s[ix] = s[iy];
    s[iy] = t;
    ++ix;
    --iy;
  }
}
```

Attacked Code Snippet

- Similarity with Query based on CodeT5+: 0.6544 (↑ 0.2694)
- Similarity with Query based on OASIS: 0.7676 (↑ 0.1616)
- Similarity with Query based on Nomic-embed-code: 0.5072 (↑ 0.4036)

```
void copies_string (unsigned char * reference, int
    ↪ content)
{
  int     assignment, iy;
  unsigned char output;

  assignment = 0;
  iy = content - 1;
  while ( assignment < iy) {
    output     = reference[ assignment];
    reference[ assignment] = reference[iy];
    reference[iy] = output;
    ++ assignment;
    --iy;
  }
}
```

Attacked Code Snippet (w/o gradient)

- Similarity with Query based on CodeT5+: 0.5427 (↑ 0.1576)
- Similarity with Query based on OASIS: 0.6856 (↑ 0.0796)
- Similarity with Query based on Nomic-embed-code: 0.3424 (↑ 0.2387)

```
void output_string (unsigned char * content, int len)
{
  int     assignment, iy;
  unsigned char takes;

  assignment = 0;
  iy = len - 1;
  while ( assignment < iy) {
    takes     = content[ assignment];
    content[ assignment] = content[iy];
    content[iy] = takes;
    ++ assignment;
    --iy;
  }
}
```

Attacked Code Snippet (w/o style preservation)

- Similarity with Query based on CodeT5+: 0.5845 (↑ 0.1995)
- Similarity with Query based on OASIS: 0.7493 (↑ 0.1434)
- Similarity with Query based on Nomic-embed-code: 0.4364 (↑ 0.3328)

```
void modificationConstantstringstd (unsigned char *
    ↪ reference, int len)
{
  int     constant, iy;
  unsigned char output;

  constant = 0;
  iy = len - 1;
  while ( constant < iy) {
    output     = reference[ constant];
    reference[ constant] = reference[iy];
    reference[iy] = output;
    ++ constant;
    --iy;
  }
}
```

Attacked Code Snippet (w/o query token similarity)

- Similarity with Query based on CodeT5+: 0.5256 (↑ 0.1405)
- Similarity with Query based on OASIS: 0.6942 (↑ 0.0883)

- Similarity with Query based on Nomic-embed-code: 0.2849 (↑ 0.1812)

```
void Compile_copy (unsigned char * Expressions, int
  ↪ getConstant)
{
  int     begin, iy;
  unsigned char iss;

  begin = 0;
  iy = getConstant - 1;
  while ( begin < iy) {
    iss     = Expressions[ begin];
    Expressions[ begin] = Expressions[iy];
    Expressions[iy] = iss;
    ++ begin;
    --iy;
  }
}
```